

UNDERSTANDING MULTI-TEACHER ON-POLICY DISTILLATION: FROM GRADIENTS TO CAPABILITIES

Anonymous authors

Paper under double-blind review

ABSTRACT

Multi-teacher on-policy distillation transfers domain expertise into a shared student. We study how loss averaging, optimizer history, and vocabulary supervision shape this transfer, using Qwen3-1.7B and four domain specialists, with additional SmoLLM3-3B measurements. Token and response averaging produce different gradients whose update directions become more similar after applying saved Adam moments; final mean scores are close. Training changes most parameters, although low-precision storage hides many of these changes. Adam’s accumulated moments also account for much of the alignment between teacher-specific updates: subtracting the zero-gradient step reduces their mean cosine from above 0.83 to below 0.01 on domain-specific inputs. Expanding supervision from a sampled token to the top-64 intersection improves final mathematics accuracy and lowers science accuracy, with relative gains changing during training. Top-64 closely approximates the full-vocabulary gradient direction in Qwen, while SmoLLM retains appreciable gradient error. Together, these findings connect local update measurements to task-dependent learning trajectories and identify optimizer history and numerical precision as central to interpreting distillation updates.

1 INTRODUCTION

Multi-teacher on-policy distillation (MOPD) transfers specialized capabilities into a single model (Xiaomi LLM-Core Team, 2026; NVIDIA, 2026; Kimi Team et al., 2026; DeepSeek-AI et al., 2026). The student generates responses, and a teacher assigned to each prompt domain supplies token-level supervision. Improving several capabilities requires deciding how much each domain and response contributes to the loss, and how many next-token alternatives the teacher supervises.

Recent work establishes both the potential of MOPD and its sensitivity to these choices. Ma et al. (2026) study capability integration with policy-gradient and top- k objectives. Open-MOPD addresses uneven transfer through domain token balancing, performance-gap weighting, and refreshed rewards (Gao et al., 2026). The Nemotron 3 Ultra report finds no advantage for top- k or full-vocabulary matching over sampled-token supervision in preliminary experiments (NVIDIA, 2026, Section 3.3.5). Other studies report parameter changes concentrated in a small set of stored weights (Yu et al., 2026; Shen et al., 2026). How these observations connect loss construction, optimizer updates, and task performance remains open.

Our study follows the path from loss construction to gradients, parameter updates, and task performance. The main experiments train Qwen3-1.7B with teachers for mathematics, code, instruction following, and science. We vary loss averaging and vocabulary supervision during training, and compare gradients and optimizer steps. Three findings emerge:

1. **Adam brings different normalization gradients closer in update direction (Section 3).** Weighting responses by length or equally produces gradients with mean cosine 0.68; Adam raises the update cosine to 0.96. The averaging rules reach similar final mean scores and follow different task-level learning curves.
2. **Precision and optimizer history strongly shape measured update sparsity and alignment (Section 4).** Training changes most FP32 weights, while BF16 rounding leaves a small fraction visibly changed. Teacher-specific steps are strongly aligned through shared

Adam moments; subtracting the zero-gradient step leaves near-zero mean alignment on domain-specific inputs.

3. **Broader vocabulary supervision produces task-dependent gains (Section 5).** Top-64 supervision improves final mathematics accuracy and lowers science accuracy relative to sampled-token supervision. Its mean advantage narrows as training proceeds. The losses produce similar update concentration.

Additional SmolLM3-3B measurements show that top-64 retains high average student probability while omitting substantial mass on some inputs. Enlarging the candidate set improves gradient approximation, with residual error relative to full-vocabulary supervision. Section 6 connects these observations to checkpoint selection: individual tasks can regress during training and recover by the endpoint, and development-set rankings can change on separate test questions. These results motivate tracking per-task learning curves alongside gradients and parameter changes.

2 PRELIMINARIES

Multi-teacher on-policy distillation (MOPD). For domain d , the student generates $y \sim p_\theta(\cdot | x)$ for $x \sim \mathcal{D}_d$, and a frozen teacher q_d supplies probabilities at each prefix $h_t = (x, y_{<t})$ (Gao et al., 2026). For a rollout batch $\mathcal{B}$, response i has T_i valid tokens and mean loss $\ell_i = T_i^{-1} \sum_t \ell_{i,t}$. The objective is

$$\mathcal{L}(\theta) = \sum_{i \in \mathcal{B}} w_i \ell_i(\theta), \quad w_i \geq 0, \quad \sum_{i \in \mathcal{B}} w_i = 1. \quad (1)$$

The token loss determines the vocabulary supervision; the response weights w_i determine normalization across tokens, responses, and domains. Each update uses fresh student responses, held fixed with their masks and weights during gradient computation.

Sampled-token, top- k , and full-vocabulary supervision. Write $p_\theta(v) = p_\theta(v | h_t)$ and $q_d(v) = q_d(v | h_t)$. Full-vocabulary supervision minimizes reverse KL over $\mathcal{V}$; the policy-gradient (PG) loss scores the sampled token y_t :

$$\ell_{\text{full}} = \sum_{v \in \mathcal{V}} p_\theta(v) \log \frac{p_\theta(v)}{q_d(v)}, \quad \ell_{\text{PG}}(y_t) = -\text{sg}[\log q_d(y_t) - \log p_\theta(y_t)] \log p_\theta(y_t) \quad (2)$$

Here sg denotes stop-gradient. At a fixed prefix, the expected PG gradient equals the full reverse-KL gradient: differentiating Equation 2 gives $\sum_v [\log(p_\theta(v)/q_d(v)) + 1] \nabla_\theta p_\theta(v)$. The +1 term cancels because $\sum_v \nabla_\theta p_\theta(v) = 0$, leaving $\mathbb{E}_{a \sim p_\theta} [\log(p_\theta(a)/q_d(a)) \nabla_\theta \log p_\theta(a)]$.

Top- k supervision uses $I = S \cap T$, where S, T are the student and teacher top- k sets, computed at rollout and held fixed. Its loss is

$$\ell_{\text{top-}k} = - \sum_{v \in I} \text{sg} \left[\frac{p_\theta(v)}{\sum_{u \in S} p_\theta(u)} (\log q_d(v) - \log p_\theta(v)) \right] \log p_\theta(v). \quad (3)$$

Probabilities use the full-vocabulary softmax; detached weights are normalized over S , and an empty intersection gives zero loss. I16 and I64 denote $k = 16$ and 64. We call the number of supervised vocabulary tokens the *supervision density*: one for PG, $|I| \leq k$ for intersection supervision, and $|\mathcal{V}|$ for full supervision.

Intersection supervision is a biased gradient surrogate. For a fixed set A , write $Z_A = \sum_{v \in A} p_\theta(v)$ and $g_A = \sum_{v \in A} \log(p_\theta(v)/q_d(v)) \nabla_\theta p_\theta(v)$. Unlike the full-vocabulary cancellation, $\nabla_\theta Z_I$ is generally nonzero. Thus directly truncating the KL objective gives $\nabla_\theta \sum_{v \in I} p_\theta(v) \log(p_\theta(v)/q_d(v)) = g_I + \nabla_\theta Z_I$. Equation 3 instead uses detached coefficients, so its gradient and discrepancy from full KL are

$$\nabla_\theta \ell_{\text{top-}k} = \frac{g_I}{Z_S}, \quad \nabla_\theta \ell_{\text{top-}k} - \nabla_\theta \ell_{\text{full}} = \left(\frac{1}{Z_S} - 1 \right) g_I - g_{\mathcal{V} \setminus I}. \quad (4)$$

I64 therefore omits contributions outside I and rescales those retained by $1/Z_S$; it is neither an unbiased full-KL gradient estimator nor the gradient of the directly truncated KL objective. The missing +1 term follows from stop-gradient, not cancellation on I . These differences mean that PG-I64 comparisons change the gradient surrogate as well as supervision density.

Table 1: **Task gains and method rankings vary across domains and evaluation sets.** Joint-training scores (%) after 500 updates; development and separate-test means weight domains equally. Development tasks are MATH-500, LiveCodeBench v6, IFBench, and GPQA Diamond. PG: sampled-token supervision; I16/I64: top-16/top-64 intersection supervision. DR/DT: equal domain weights with response/token averaging; GT: global token averaging. Δ is the development-mean difference from PG-DR/Adam (percentage points); brackets give model-based paired 95% intervals (Appendix A.3).

Configuration	Math	Code	IF	Science	Dev. mean	Test mean	Δ [95% interval]
Initial student	68.80	16.41	18.33	32.20	33.94	38.60	-4.68 [-6.42, -2.93]
<i>Adam</i>							
PG-DR	73.20	18.27	27.00	35.97	38.61	43.55	Reference
PG-DT	73.20	19.24	27.00	34.96	38.60	43.51	-0.01 [-1.45, 1.43]
PG-GT	76.20	17.92	27.33	32.94	38.60	44.51	-0.01 [-1.44, 1.42]
I16-DR	75.40	17.75	27.00	35.46	38.90	44.56	+0.29 [-1.15, 1.73]
I64-DR	75.80	18.30	27.33	34.71	39.04	44.46	+0.43 [-1.01, 1.86]
I64-DT	74.00	18.86	28.67	35.22	39.19	43.68	+0.58 [-0.87, 2.02]
I64-GT	74.20	19.80	28.67	33.41	39.02	44.97	+0.41 [-1.03, 1.85]
<i>SGD</i>							
PG-DR	76.40	19.53	28.33	35.59	39.96	43.48	+1.35 [-0.09, 2.79]
PG-DT	75.20	20.31	28.33	33.90	39.44	43.90	+0.83 [-0.61, 2.26]
PG-GT	75.00	19.26	25.33	34.71	38.58	42.97	-0.04 [-1.47, 1.40]

Domain weighting and loss normalization. Let $\mathcal{B}_d$ contain responses from domain d , and let $\lambda_d \geq 0$ with $\sum_d \lambda_d = 1$ be the target domain loss weights. For nonempty responses and domains, the three rules assign response $i \in \mathcal{B}_d$ weights

$$w_i^{\text{GT}} = \frac{T_i}{\sum_{j \in \mathcal{B}} T_j}, \quad w_i^{\text{DT}} = \frac{\lambda_d T_i}{\sum_{j \in \mathcal{B}_d} T_j}, \quad w_i^{\text{DR}} = \frac{\lambda_d}{|\mathcal{B}_d|}. \quad (5)$$

Global-token (GT) averaging weights domains by their response-token shares. Domain-token (DT) and domain-response (DR) averaging fix each domain’s total weight to λ_d : DT weights responses by length, while DR weights their mean losses equally. Domain loss weights are distinct from prompt sampling proportions.

Update similarity. Let u_d be the parameter change from one step on domain d ’s mean loss. Each compared step starts from identical student weights and optimizer state, including Adam’s moments. For nonzero updates, their cosine is $C_{de} = (u_d^\top u_e) / (\|u_d\| \|u_e\|)$.

3 LOSS NORMALIZATION AND TASK PERFORMANCE

We compare the three averaging rules when training Qwen3-1.7B with four RL-specialized teachers derived from the same base model, covering mathematics, code, instruction following, and science. Table 1 summarizes scores after 500 updates, with means weighting the four domains equally. Full training and evaluation details, including uncertainty estimates, are in Appendix A.

3.1 RESPONSE LENGTH CHANGES THE CONTRIBUTION OF EACH DOMAIN AND RESPONSE

Longer responses receive more weight when the loss is averaged over all tokens. In our training batches, mathematics accounts for about three times as many response tokens as instruction following, even though the two domains receive equal numbers of prompts (Figure 1a). Global token averaging therefore gives mathematics a larger share of the loss. Averaging each domain separately fixes its total contribution; averaging response losses equally also removes the extra weight assigned to long responses.

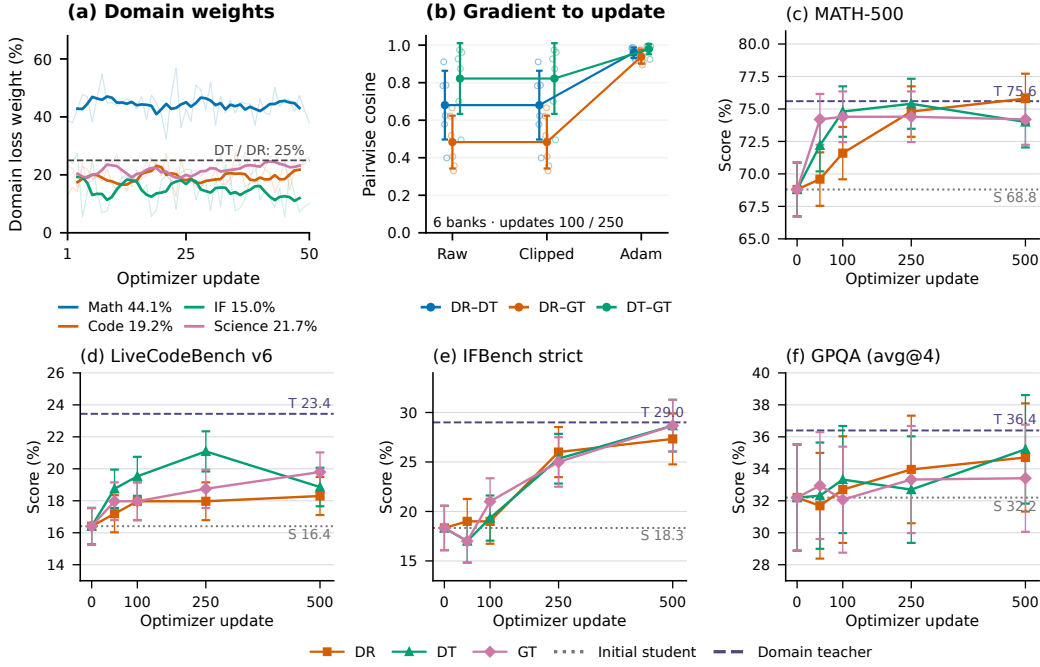


Figure 1: **Loss averaging changes gradient directions; Adam produces more aligned updates and similar final mean scores.** (a) Domain shares under global token averaging (GT). (b) Gradient and update cosines for global token, within-domain token (DT), and within-domain response (DR) averaging; points and bars show batch means and standard deviations. (c–f) Task learning curves under top-64 intersection supervision (I64); S/T mark the initial student/teacher; error bars show model-based question-sampling standard deviations.

The remaining effect of length within a domain has a simple form. For response gradients $g_i = \nabla_{\theta} \ell_i$ and domain means $\bar{g}_d, \bar{T}_d$, the token-weighted gradient is

$$\frac{\sum_{i \in \mathcal{B}_d} T_i g_i}{\sum_{i \in \mathcal{B}_d} T_i} = \bar{g}_d + \frac{\text{Cov}_d(T, g)}{\bar{T}_d}. \quad (6)$$

Token averaging retains the length–gradient covariance term; response averaging uses $\bar{g}_d$. Correcting each domain’s global-token contribution by its target weight divided by its observed token share gives domain-token averaging exactly, as in Open-MOPD (Gao et al., 2026). This correction preserves within-domain length weighting.

3.2 TRAINED ADAM MOMENTS ALIGN UPDATES, WITH SIMILAR FINAL SCORES

Different gradients produce closely aligned Adam steps. Equal response weighting and within-domain token weighting produce gradients with mean cosine 0.68. Adam raises their update cosine to 0.96 (Figure 1b). Comparisons with global token averaging show the same pattern.

All rules start from the same saved optimizer state: Adam mixes each gradient with a shared first-moment history and rescales coordinates through its second moment (Kingma & Ba, 2015). High update alignment therefore need not imply similar gradients. This effect depends on optimizer state: zero-initialized Adam moments instead reduce alignment in the SmolLM diagnostic (Appendix H.3).

Clipping preserves gradient direction; optimizer history shapes the step. Response averaging triggers clipping more often than token averaging, but global norm clipping preserves pairwise gradient cosines (Figure 2a). Resetting Adam’s moments changes step norms and held-out loss



Figure 2: **Response averaging triggers more clipping; Adam history shapes step norms and one-step loss changes.** (a) Retained gradient fraction and clipping rates for response (DR), within-domain token (DT), and global token (GT) averaging. (b) Step norms after resetting Adam’s first moment or both moments, retaining the step count. (c) Held-out reverse-KL changes (lower is better). Open points are the three batches per checkpoint; filled points and bars show their means and standard deviations. SGD steps match the corresponding Adam step norm.

changes; norm-matched SGD steps separate the rules more strongly (panels b,c). Aggregate one-step improvements can also coexist with domain-level regressions (Appendix G.6).

Final means are close under Adam despite different task trajectories. With I64 supervision, the three rules finish within 0.17 percentage points in mean score, although their learning curves and task rankings differ (Figure 1c–f). With sampled-token supervision, they are also nearly tied under Adam, whereas SGD yields an observed spread of about 1.4 points, with response averaging highest (Table 1).

4 PARAMETER-UPDATE SPARSITY AND TEACHER ALIGNMENT

We examine numerical precision and shared optimizer state when interpreting concentrated checkpoint changes (Yu et al., 2026) and teacher-update alignment.

4.1 BF16 ROUNDING HIDES WIDESPREAD FP32 CHANGES

Joint training changes about 97% of FP32 master weights. After BF16 rounding, only 7–9% of weights differ visibly from initialization (Figure 3a). Small updates are widespread in FP32 and disappear when the weights are stored in BF16.

The precision effect also extends to the concentration of update magnitude. About one third of FP32 coordinates carry 90% of squared cumulative change, compared with about 4% of BF16 coordinates.

Loss averaging provides a further comparison between parameter changes and performance. The averaging rules produce different fractions of changed BF16 weights while reaching similar final mean scores.

4.2 SHARED ADAM MOMENTS ALIGN TEACHER-SPECIFIC UPDATES

Teacher-specific steps are strongly aligned by Adam. On responses from each teacher’s own domain, mean update cosines exceed 0.83 under both sampled-token and top-64 supervision. Resetting the first moment while retaining the second, or replacing Adam with norm-matched SGD, reduces the mean cosine to near zero (Figure 3b). The accumulated first moment accounts for much of the alignment.

Adam’s first moment can move parameters even with zero current gradient. To measure the change in a step induced by the current teacher gradient, we subtract this baseline. For update function $U(g; H)$ and saved state H ,

$$\delta U_d = U(g_d; H) - U(0; H). \quad (7)$$

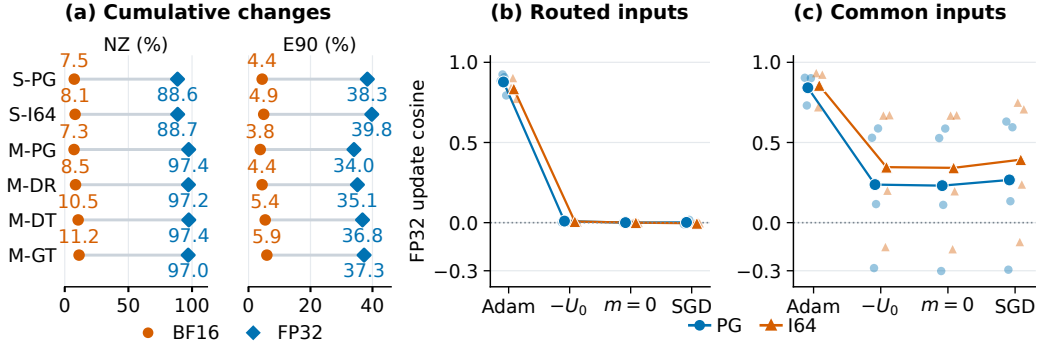


Figure 3: **BF16 rounding hides widespread parameter changes, and shared Adam moments align teacher-specific steps.** (a) Fractions of changed coordinates (NZ) and coordinates carrying 90% of squared change (E90), measured in FP32 and BF16. (b,c) Update cosines on domain-specific/shared inputs; $-U_0$ subtracts the zero-gradient step and $m = 0$ resets the first moment. Small/large points show batch/overall means. S/M denote mathematics-only/joint training and PG/I64 denote sampled-token/top-64 supervision. M-DR/DT/GT use top-64 with response/within-domain token/global token averaging; other rows use response averaging.

After this subtraction, the mean cosine between teacher-specific step differences falls below 0.01 on domain-specific inputs. When all teachers score the same prefixes, the residual cosine is larger, about 0.24–0.35 (Figure 3c). Teacher-specific alignment therefore depends on both shared optimizer history and shared inputs: removing the zero-gradient component reveals much greater directional separation across domains.

5 VOCABULARY SUPERVISION: PARAMETER UPDATES AND TASK PERFORMANCE

Expanding supervision from one sampled token to a larger vocabulary set changes gradient approximation and task-level learning. We compare sampled-token supervision with the student–teacher top-64 intersection during mathematics-only and joint training. Full-vocabulary supervision supplies a reference for the gradient and single-step comparisons.

5.1 BROADER SUPERVISION PRODUCES TASK-DEPENDENT GAINS

The mathematics advantage develops during training. In mathematics-only training, sampled-token supervision leads top-64 by 2.8 accuracy points at update 100; top-64 leads by 2.2 points at update 500 (Figure 4c). Joint training also ends with a mathematics gain from top-64 supervision, accompanied by a science loss: final differences are +2.6 points on mathematics and -1.3 on GPQA. Code and instruction-following differences are smaller.

Top-64’s joint-training mean advantage narrows from 1.69 points at update 250 to 0.43 at update 500. These trajectories show that the observed benefit depends on the task and the stage of training.

Larger candidate sets redistribute gains across tasks. Top-16 and top-64 supervision reach development means of 38.90% and 39.04%, and separate-test means of 44.56% and 44.46% (Table 1). On the separate tests, top-64 gains 1.96 mathematics points and loses 2.35 code points relative to top-16.

5.2 LARGER CANDIDATE SETS IMPROVE GRADIENT APPROXIMATION

Truncating vocabulary support omits probability mass and changes token weights (Anshumann et al., 2025; Huang & Wei, 2026). We measure coverage as the student probability retained by the student–teacher candidate intersection, $c_k(h) = \sum_{v \in S \cap T} p_\theta(v | h)$, and compare gradients.

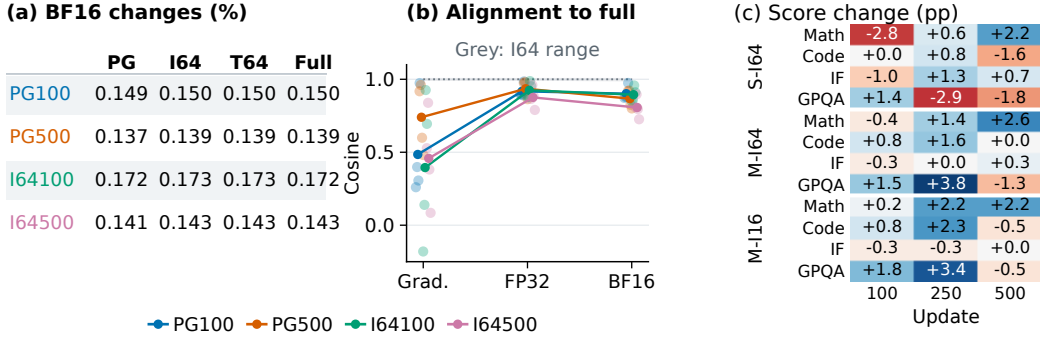


Figure 4: **Top-64 supervision closely approximates the full-vocabulary gradient in Qwen and produces task-dependent learning gains.** (a) Fraction of changed BF16 weights for each local loss at fixed checkpoints. (b) Gradient and update cosines to full-vocabulary supervision. (c) Score differences from sampled-token supervision over training. PG: sampled token; I16/I64: student-teacher top-16/top-64 intersection; T64: teacher top-64. S/M denote mathematics-only/joint training; checkpoint numbers are training updates.

Top-64 closely matches the full-vocabulary direction in Qwen. The intersection retains over 99.9% of student probability on average, and its gradient direction is close to that of full-vocabulary supervision (Figure 4b). Averaging 1, 16, and 64 token samples raises the mean gradient cosine from 0.54 to 0.87 to 0.97; top-64 exceeds 0.999 (Figure 6a). This sampling control identifies finite-sample variation as one source of the directional difference.

Lower-coverage inputs retain appreciable gradient error in SmoLLM. SmoLLM3-3B measurements use the MixSFT initialization and its student after 500 updates of top-64 training with response averaging and three teachers for mathematics, code, and instruction following (Gao et al., 2026). Mean top-64 coverage is 97.42% initially and 98.56% after training; 8.42% and 3.65% of weighted prefixes retain less than 90% mass (Figure 5a,b). Larger candidate sets reduce this low-coverage tail.

We compare gradients on typical prefixes, high-entropy prefixes, and prefixes with top-64 coverage below 99%. Top-64 has 2.16–24.54% relative gradient error against full-vocabulary supervision. Top-4 and top-8 produce larger errors in the evaluated batches (Figure 5d). Here relative error is the norm of the gradient difference divided by the full-vocabulary gradient norm, capturing both direction and magnitude. Larger candidate sets improve the approximation, with residual error at top-64.

SmoLLM capability scores also reveal changes below the aggregate. Sampled-token training with SGD raises MATH-500 accuracy from 50.60% to 53.80%. On AIME 2025, the initialization, the top-64/domain-token Adam student, and the sampled-token/response-averaged SGD student each solve four of 30 questions with eight samples per question; only one solved question is common to all three.

5.3 VOCABULARY SUPERVISION LEAVES SINGLE-STEP CONCENTRATION NEARLY UNCHANGED

Sampled-token, top-64, teacher-top-64, and full-vocabulary losses produce nearly identical single-step concentration (Figure 4a). This similarity across vocabulary supports is consistent with concentrated stored-weight changes under dense distillation (Yu et al., 2026).

The teacher-top-64 reference uses a fixed teacher top-64 set T and full-vocabulary probabilities:

$$\ell_{T64} = \sum_{v \in T} \left[p_{\theta}(v) \log \frac{p_{\theta}(v)}{q_d(v)} - p_{\theta}(v) + q_d(v) \right]. \quad (8)$$

The $-p_{\theta}(v)$ correction cancels the $+1$ derivative term on this restricted support; $q_d(v)$ is constant with respect to the student and makes each summand zero when $p_{\theta}(v) = q_d(v)$. Hence $\nabla_{\theta} \ell_{T64} =$

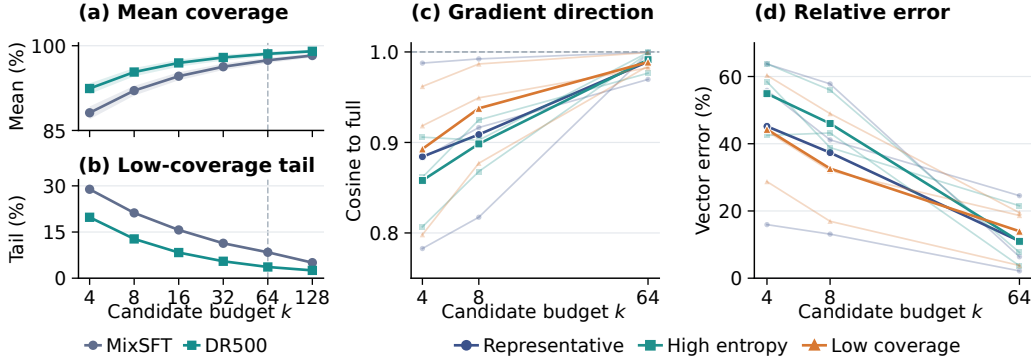


Figure 5: **Larger candidate sets improve coverage and gradient approximation in SmolLM.** (a,b) Mean retained student probability and fraction of inputs below 90% coverage; bands show response-cluster 95% intervals. Dashed lines mark top-64. (c,d) Gradient cosine and relative error against full-vocabulary supervision at the trained student. Groups comprise typical inputs, uncertain next-token predictions, and inputs with lower top-64 coverage; thin lines show batches and bold lines their means. MixSFT/DR500 denote the initialization/trained student.

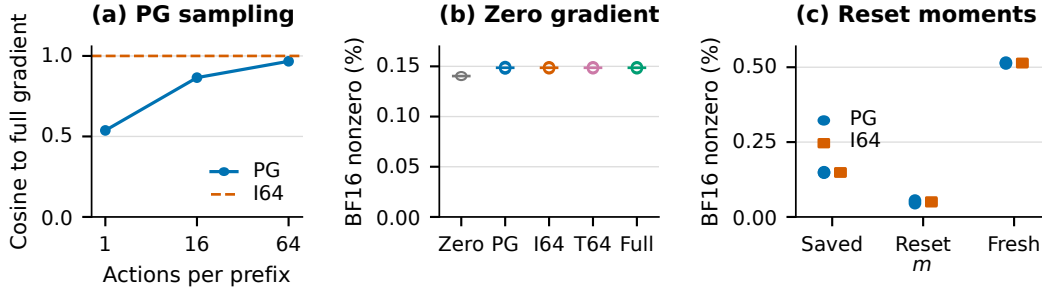


Figure 6: **Averaging more token samples improves gradient alignment; Adam history contributes strongly to BF16 changes.** (a) Gradient cosine to full-vocabulary supervision; I64 uses the top-64 intersection. (b,c) Fractions of changed BF16 weights with zero current gradient or reset moments. PG denotes sampled-token supervision; reset- m keeps the second moment and step count, and fresh Adam resets both moments and the count. Points show batches and bars show means.

g_T , using the notation of Section 2. T64 omits terms outside T but has no $1/Z_S$ rescaling, whereas I64 uses g_I/Z_S . Neither generally recovers the full-KL gradient. T64 and full KL are single-step references; end-to-end training compares PG, I16, and I64.

Shared optimizer history contributes strongly to the similar concentration. Even a zero current gradient changes about 0.14% of BF16 parameters in the same comparison, close to the fraction changed by the supervised losses. Resetting Adam’s moments changes the amount of BF16 writeback (Figure 6b,c).

Over training, student parameters, responses, and optimizer moments evolve together. Top-64 and sampled-token supervision consequently accumulate different BF16 displacements (Section 4), alongside the task-dependent learning trajectories above.

6 TASK-LEVEL MONITORING AND CHECKPOINT SELECTION

Separate test questions assess transfer beyond development rankings. We evaluate 256 additional questions per domain, using Skywork mathematics and TACO code pools, Nemotron instruction-following data, and the natural-science portion of Nemotron knowledge validation. Exact and lexical overlap with the training and known evaluation pools is removed. Scoring uses

answer verification, unit tests, instruction constraints, and multiple-choice accuracy, with the same decoding scheme as development evaluation. Table 1 reports its means for all ten final students.

Intermediate checkpoints reveal temporary domain regressions. With sampled-token supervision and response averaging under Adam, science accuracy changes from 32.20% initially to 28.03% at update 50, 30.18% at update 250, and 35.97% at update 500. Tracking individual domains captures this temporary regression and its recovery.

A domain-retention constraint selects the same checkpoints as the mean score. We compare the highest development mean with the highest mean subject to

$$s_d(\theta) \geq s_d(\theta_0) - \epsilon_d \quad \text{for all } d, \quad \epsilon_d = 2 \text{ percentage points}, \quad (9)$$

where s_d is the domain score and θ_0 the initial student. Both rules retain the initialization unless the mean improves by at least one point.

Both rules select update 500 in nine trajectories and update 250 for sampled-token training with domain-token averaging under Adam. The earlier checkpoint scores 43.14% on the separate test, compared with 43.51% at update 500.

Method rankings vary with evaluation questions. For sampled-token supervision with response averaging, SGD leads Adam by 1.35 mean points on development benchmarks; separate-test means are 43.48% and 43.55%, respectively. A further sensitivity check removes four MATH-500 questions with known training-pool matches: four-domain means change by 0.09–0.16 points, reordering some near-tied methods while preserving the highest observed mean for response-averaged SGD. Together, these results support reading aggregate rankings alongside per-task trajectories and the evaluation questions that produced them.

7 RELATED WORK

Distillation objectives and vocabulary truncation. MiniLLM uses reverse KL (Gu et al., 2024); Generalized Knowledge Distillation (GKD) combines student-generated sequences with alternative divergences (Agarwal et al., 2024); f -DISTILL derives word-level losses from sequence-level f -divergences (Wen et al., 2023). Sparse Logit Sampling analyzes top- k caching bias (Anshumann et al., 2025), while Tail-Aware OPD restores tail-mass information lost by top- k renormalization (Huang & Wei, 2026). Our intersection loss retains full-softmax log ratios with detached student-top- k weights. Section 5 compares its coverage, gradient error, updates, and task scores with sampled-token and full-vocabulary supervision.

Multi-teacher distillation and loss normalization. MOPD integrates specialized teachers on student-generated responses (Ma et al., 2026); Open-MOPD adds domain token balancing and adaptive weighting (Gao et al., 2026). DAPO uses token-level averaging (Yu et al., 2025), and Dr. GRPO analyzes length and reward-normalization bias (Liu et al., 2025). We separate domain weighting from within-domain response weighting: Open-MOPD’s token-share correction gives DT for fixed batches and domain weights, while DR also removes response-length weighting.

Post-training parameter geometry. Mukherjee et al. (2025) study small parameter subsets supporting RL fine-tuning; Yu et al. (2026) report concentrated distillation updates and overlap between separately trained students. Shen et al. (2026) examine OPD geometry and response-position selection. We examine teacher alignment, distinguish FP32 from BF16 changes, and vary vocabulary support.

8 DISCUSSION

The path from a distillation loss to task performance passes through optimizer history, numerical precision, and an evolving training trajectory. In our comparisons, Adam aligns directionally different gradients, and BF16 rounding hides widespread FP32 changes. Broader vocabulary supervision improves gradient approximation and produces task-dependent learning gains, including

a mathematics–science trade-off and changing advantages over time. These findings give complementary roles to the measurements: gradients describe the immediate training signal, parameter changes show its interaction with the optimizer and precision, and per-task learning curves reveal the resulting capability gains and regressions.

AI USE STATEMENT

We used a large language model for minor editorial assistance, including correcting grammar and typo and refining formatting. It did not contribute to the development of research ideas, the analysis, or the claims.

ETHICS STATEMENT

We ensure that the dataset used in this study is entirely free from discrimination, bias, exploitation, human rights violations, and any other ethical concerns.

REPRODUCIBILITY STATEMENT

To ensure reproducibility of our results, we provide detailed documentation of our experimental setup and methodology in the main paper and appendix. The code for data, evaluation, and analysis will be open-sourced upon publication.

REFERENCES

- Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2306.13649>.
- Anshumann, Mohd Abbas Zaidi, Akhil Kedia, Jinwoo Ahn, Taehwak Kwon, Kangwook Lee, Haejun Lee, and Joohyung Lee. Sparse logit sampling: Accelerating knowledge distillation in LLMs. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 18085–18108. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.885. URL <https://aclanthology.org/2025.acl-long.885/>.
- DeepSeek-AI, Anyi Xu, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chenchen Ling, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chengyu Hou, Chenhao Xu, Chenze Shao, Chong Ruan, Conner Sun, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Donghao Li, Dongjie Ji, Erhang Li, Fang Wei, Fangyun Lin, Fangzhou Yuan, Feiyu Xia, Fucong Dai, Guangbo Hao, Guanting Chen, Guoai Cao, Guolai Meng, Guowei Li, Han Yu, Han Zhang, Hanwei Xu, Hao Li, Haofen Liang, Haoling Zhang, Haoming Luo, Haoran Wei, Haotian Yuan, Haowei Zhang, Haowen Luo, Haoyu Chen, Haozhe Ji, Hengqing Zhang, Honghui Ding, Hongxuan Tang, Huanqi Cao, Huazuo Gao, Hui Qu, Hui Zeng, J Yang, JQ Zhu, Jia Luo, Jia Song, Jia Yu, Jialiang Huang, Jialu Cai, Jian Liang, Jiangting Zhou, Jiasheng Ye, Jiashi Li, Jiaxin Xu, Jiewen Hu, Jieyu Yang, Jin Chen, Jin Yan, Jingchang Chen, Jingli Zhou, Jingting Xiang, Jingyang Yuan, Jingyuan Cheng, Jingzi Zhou, Jinhua Zhu, Jiping Yu, Joseph Sun, Jun Ran, Junguang Jiang, Junjie Qiu, Junlong Li, Junmin Zheng, Junxiao Song, Kai Dong, Kaige Gao, Kang Guan, Kexing Zhou, Kezhao Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Wang, Leyi Xia, Li Zhang, Liang Zhao, Lihua Guo, Lingxiao Luo, Linwang Ma, Linyan Zhu, Litong Wang, Liyu Cai, Liyue Zhang, Longhao Chen, MS Di, MY Xu, Max Mei, Miaojuan Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Mingxu Zhou, Minmin Han, Ning Wang, Panpan Huang, Panpan Wang, Peixin Cong, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qingyang Li, Qinyu Chen, Qiushi Du, Qiwei Jiang, Rui Tian, Ruifan Xu, Ruijie Lu, Ruiling Xu, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runqian Chen, Runqiu Yin, Runxin Xu, Ruomeng Shen, Ruoyu Zhang, Ruyi Chen, SH Liu, Shanghao Lu, Shangmian Sun, Shangyan Zhou, Shanhuang Chen, Shaofei Cai, Shaoheng Nie, Shaoqing Wu, Shaoyuan Chen, Shengding Hu, Shengyu Liu, Shiqiang Hu, Shirong Ma, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, Shuying Yu, Songyang Zhou, Tao Ni, Tao Yun, Tian Jin, Tian Pei, Tian Ye, Tianle Lin, Tianran Ji, Tianyi Cui, Tianyuan Yue, Tingting Yu, Tun Wang, W Zhang, WL Xiao, Wangding Zeng, Wei An, Weilin Zhao, Wen Liu, Wenfeng Liang, Wenjie Pang, Wenjing Luo, Wenjing Yao, Wenjun Gao, Wenkai Yang, Wenlve Huang, Wenqing Hou, Wentao Zhang, Wenting Ma, Xi Gao, Xiang He, Xiangwen Wang, Xianzu Wang,

- Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaokang Zhang, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingchen Liu, Xingkai Yu, Xingyou Li, Xinyu Yang, Xinyu Zhang, Xu Chen, Xuanyu Wang, Xuecheng Su, Xueyin Chen, Xuheng Lin, Xuwei Fu, YC Yan, YQ Wang, YW Ma, Yanfeng Luo, Yang Zhang, Yanhong Xu, Yanru Ma, Yanwen Huang, Yao Li, Yao Li, Yao Xu, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Qian, Yi Shao, Yi Yu, Yichao Zhang, Yifan Ding, Yifan Shi, Yijia Wu, Yiliang Xiong, Yiling Ma, Ying He, Ying Tang, Ying Zhou, Yingjia Luo, Yinmin Zhong, Yishi Piao, Yisong Wang, Yixiang Zhang, Yixiao Chen, Yixuan Tan, Yixuan Wei, Yiyang Ma, Yiyuan Liu, Yonglun Yang, Yongqiang Guo, Yongtong Wu, Yu Wu, YuKun Li, Yuan Cheng, Yuan Ou, Yuanfan Xu, Yuanhao Li, Yuduan Wang, Yuehan Yang, Yuer Xu, Yuhao Meng, Yuheng Zou, Yukun Zha, Yunfan Xiong, Yupeng Chen, Yuping Lin, Yuqian Cao, Yuqian Wang, Yushun Zhang, Yuting Yan, Yutong Lin, Yuxian Gu, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuxuan Zhou, Yuyang Zhou, Yuzhen Huang, ZF Wu, Zehao Wang, Zehua Zhao, Zehui Ren, Zekai Zhang, Zhangli Sha, Zhe Fu, Zhe Ju, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zheren Gao, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhixian Huang, Zhixuan Chen, Zhiyu Wu, Zhizhou Ren, Zhongyu Wu, Zhuoshu Li, Zhuping Zhang, Zian Xu, Zihao Wang, Zihua Qu, Zihui Gu, Zijia Zhu, Zilin Li, Zipeng Zhang, Ziwei Xie, Ziyi Gao, Ziyi Wan, Zizheng Pan, and Zongqing Yao. Deepseek-v4: Towards highly efficient million-token context intelligence, 2026. URL <https://arxiv.org/abs/2606.19348>.
- Huan-ang Gao, Haohan Chi, Yong Yan, Shiyuan Feng, Hanlin Wu, Zheng Jiang, Bingxiang He, Wei-Ying Ma, Ya-Qin Zhang, and Hao Zhou. Open-MOPD: Diagnosing and fixing capability imbalance in multi-teacher on-policy distillation, 2026. URL <https://arxiv.org/abs/2608.19098v1>.
- Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. MiniLLM: Knowledge distillation of large language models. In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2306.08543v4>.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems*, 2021. doi: 10.48550/arXiv.2103.03874. URL <https://arxiv.org/abs/2103.03874v2>.
- Huipeng Huang and Hongxin Wei. Tail-aware top- k on-policy distillation, 2026. URL <https://arxiv.org/abs/2608.14728>.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code, 2024. URL <https://arxiv.org/abs/2403.07974v2>.
- Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, M. C., Jianfeng Cai, Xinyuan Cai, Peizhou Cao, Yuxuan Cao, Ziwei Chai, Y. Charles, H. S. Che, Guanduo Chen, Guangyu Chen, Guanzheng Chen, Huarong Chen, Jia Chen, Jianlong Chen, Jun Chen, Kexin Chen, Peng Chen, Ruijue Chen, Wentao Chen, Xin Chen, Yang Chen, Yanru Chen, Yifei Chen, Yingjiang Chen, Yuankun Chen, Yujie Chen, Yutian Chen, Zhirong Chen, Dazhi Cheng, Yean Cheng, Jialei Cui, Jingbing Cui, Anqi Dai, Jiaqi Deng, Hao Ding, Rui Ding, Shaofeng Ding, Mengfan Dong, Mengnan Dong, Yuhao Dong, Yuxin Dong, Angang Du, Chenzhuang Du, Dikang Du, Jusen Du, Yulun Du, Yu Fan, Jing Feng, Qiulin Feng, Yichen Feng, Kelin Fu, Qiang Fu, Fuxuan Gao, Hongcheng Gao, Jingyue Gao, Tong Gao, Weijia Gao, Shangyi Geng, Jie Gong, Linhu Gong, Shengao Gong, Xiaochen Gong, Qizheng Gu, Yicheng Gu, Shuhao Guan, Haiqing Guo, Shiqi Guo, Xiang Guo, Zhengyan Guo, Beixi Hao, Wenxin Hao, Xiaoru Hao, Dailan He, Haotian He, Lehan He, Qi He, Weiran He, Xinran He, Xinyi He, Yibo He, Yunjia He, Chao Hong, Tiange Hong, Hao Hu, Jiaxi Hu, Ruikun Hu, Weiming Hu, Yangyang Hu, Zhenxing Hu, Liang Hua, Jinbin Huang, Ke Huang, Ruiyuan Huang, Siying Huang, Weixiao Huang, Yan Huang, Zhengjie Huang, Zhiqi Huang, Yulong Hui, Chaobo Jia, Yutong Jiang, Zhejun Jiang, Zuoyou Jiang, Wenyi Jin, Xinyi Jin, Yu Jing, Huanjun Kong, Guokun Lai, Aidi Li, Cheng Li, Chengyuan Li, Cong Li, Fang Li, Guanyu Li, Haoyang Li, Jia Li, Junxiong Li, Lei Li, Letian Li, Lincan Li, Weihong Li, Wentao Li, Xintong Li, Yang Li, Yishen Li, Yiwei Li, Yuxiao Li, Zhaowei Li, Zhaoxi Li, Zheming Li, Zhengxiao Li, Zhiyuan Li, Jiawei Lin, Xiaohan Lin, Yibo Lin, Zichao Lin, Ziyan Lin, Bill Liu, Boxiao Liu, Chuan Liu,

Liang Liu, Shaowei Liu, Shudong Liu, Shuran Liu, Tianwei Liu, Weizhou Liu, Yangyang Liu, Yanming Liu, Yibo Liu, Yipeng Liu, Zhengying Liu, Zhiheng Liu, Enzhe Lu, Haoyu Lu, Linqiang Lu, Tingzhan Lu, Zhiyuan Lu, Aotian Luo, G. Luo, Junyu Luo, Yifan Luo, B. Lyu, Wenzhou Lyu, Shaoguang Mao, Yuan Mei, Xin Men, Mingqing Ni, Yixuan Niu, Siyuan Pan, Shujun Peng, Zhangyang Qi, Ruoyu Qin, ZeChao Qin, Zeyu Qin, Haiquan Qiu, Jianxin Qiu, Jiezhong Qiu, Bowen Qu, Yuhao Qu, Zeyu Shang, Youbo Shao, Han Shen, Jincheng Shi, Juanfeng Shi, Lidong Shi, Shengyuan Shi, Wingchun Siu, Pengwei Song, Xiaoxi Song, Jianlin Su, Yunfeng Su, Zhaochen Su, Lin Sui, Jingsong Sun, Junyao Sun, Shaoning Sun, Shuzhe Sun, Tongyu Sun, Yujun Sun, Yunpeng Tai, Chuning Tang, Heyi Tang, Sirui Tang, Zecheng Tang, Chaoran Tian, Rongpeng Tian, Yu Tian, Wei Tu, Chensi Wang, Chuang Wang, Chunjie Wang, Dinglu Wang, Feng Wang, Hailong Wang, Haiming Wang, Hao Wang, Hao Wang, Huaqing Wang, Hui Wang, Jiayi Wang, Jinglong Wang, Jinhong Wang, Jiuzheng Wang, Linian Wang, Shaobo Wang, Shenzhi Wang, Shuyi Wang, Si Wang, Siyuan Wang, Tianfu Wang, Wenjue Wang, Xingran Wang, Xinmei Wang, Xinyuan Wang, Xusheng Wang, Yalin Wang, Yangkun Wang, Yao Wang, Yaoyu Wang, Yejie Wang, Yiqin Wang, Yucheng Wang, Yuzhi Wang, Zhaoji Wang, Zhaowei Wang, Zhengtao Wang, Zhenhao Wang, Zhongsheng Wang, Zifan Wang, Chu Wei, Ming Wei, Shouxin Wei, Zichen Wen, Fan Wu, Haoning Wu, Rucong Wu, Wenhao Wu, Xiaoxue Wu, Yingcong Wu, Yongqi Wu, Yuxin Wu, Zijian Wu, Xinglang Xian, Chenxuan Xiang, Yuye Xiang, Bocheng Xiao, Chenjun Xiao, Xin Xiao, Jin Xie, Xiaotong Xie, Yifeng Xie, Zhe Xie, Bowei Xing, Yiming Xiong, Baosheng Xu, Boyu Xu, Jiale Xu, Jianfan Xu, Jing Xu, Jinjing Xu, L. H. Xu, Qingtao Xu, Shuyao Xu, Suting Xu, Tiantian Xu, Tianxiang Xu, Weixin Xu, Xinran Xu, Yangchuan Xu, Ye Xu, Yueni Xu, Ziyao Xu, Haonan Xue, Junjie Yan, Yaoyao Yan, Fan Yang, Guangyao Yang, Hao Yang, Junwei Yang, Ruoyu Yang, Wenjie Yang, Xiaofei Yang, Xinyu Yang, Yi Yang, Yiling Yang, Ying Yang, Yuchen Yang, Zhen Yang, Zhilin Yang, Zian Yang, Zuhao Yang, Haotian Yao, Dan Ye, Haoran Ye, Wenjie Ye, Zhanbo Ye, Bohong Yin, Haoxiang Yin, Xietong Yin, Chengzhen Yu, Haozhen Yu, Longhui Yu, Shengnan Yu, Shuying Yu, Tianxiang Yu, Enming Yuan, Mengjie Yuan, Tongtian Yue, Wei Yue, Yang Yue, Dunyuan Zha, Haobing Zhan, B. H. Zhang, Dehao Zhang, Fei Zhang, Hao Zhang, Haoyuan Zhang, Huanyu Zhang, Jiapei Zhang, Jiaxuan Zhang, Jin Zhang, Kaiyi Zhang, Miaozhen Zhang, Puqi Zhang, Qinglei Zhang, Rong Zhang, Rui Zhang, Shaoshuai Zhang, Shiyi Zhang, Xiaobin Zhang, Xiaoyun Zhang, Y. Zhang, Yangkun Zhang, Ye Zhang, Yichi Zhang, Yikun Zhang, Yizhi Zhang, Yongting Zhang, Yu Zhang, Yutao Zhang, Yutong Zhang, Zheng Zhang, Zijing Zhang, Bin Zhao, Chenguang Zhao, Feifan Zhao, Jinglun Zhao, Jinxiang Zhao, Shuai Zhao, Wenshuo Zhao, Xiangyu Zhao, Xuanle Zhao, Yikai Zhao, Zijia Zhao, Haozhi Zheng, Huabin Zheng, Ruihan Zheng, Shaojie Zheng, Tengyang Zheng, Haofeng Zhong, Lei Zhong, Longguang Zhong, M. Zhou, Qian Kang Zhou, Runjie Zhou, Ruozhang Zhou, Xinyu Zhou, Yiqiao Zhou, Zaida Zhou, Jinguo Zhu, Liya Zhu, Xinhao Zhu, Yangjunfeng Zhu, Yuxuan Zhu, Zhen Zhu, Chen Zhuang, Weiyu Zhuang, and Xinxing Zu. Kimi k3: Open frontier intelligence, 2026. URL <https://arxiv.org/abs/2607.24653>.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. URL <https://arxiv.org/abs/1412.6980>.

Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding R1-Zero-like training: A critical perspective, 2025. URL <https://arxiv.org/abs/2503.20783>.

Wenhan Ma, Jianyu Wei, Liang Zhao, Hailin Zhang, Bangjun Xiao, Lei Li, Qibin Yang, Bofei Gao, Yudong Wang, Rang Li, Jinhao Dong, Zhifang Sui, and Fuli Luo. MOPD: Multi-teacher on-policy distillation for capability integration in llm post-training, 2026. URL <https://arxiv.org/abs/2606.30406v1>.

Sagnik Mukherjee, Lifan Yuan, Dilek Hakkani-Tur, and Hao Peng. Reinforcement learning finetunes small subnetworks in large language models, 2025. URL <https://arxiv.org/abs/2505.11711v2>. Version 2; NeurIPS 2025.

Sagnik Mukherjee, Lifan Yuan, Pavan Jayasinha, Dilek Hakkani-Tur, and Hao Peng. Do we need Adam? surprisingly strong and sparse reinforcement learning with SGD in LLMs, 2026. URL <https://arxiv.org/abs/2602.07729>.

- NVIDIA. Nemotron 3 Ultra: Open, efficient mixture-of-experts hybrid Mamba-Transformer model for agentic reasoning, 2026. URL <https://arxiv.org/abs/2606.15007v1>.
- Valentina Pyatkin, Saumya Malik, Victoria Graf, Hamish Ivison, Shengyi Huang, Pradeep Dasigi, Nathan Lambert, and Hannaneh Hajishirzi. Generalizing verifiable instruction following, 2025. URL <https://arxiv.org/abs/2507.02833v3>.
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level google-proof q&a benchmark, 2023. URL <https://arxiv.org/abs/2311.12022v1>.
- Zhennan Shen, Yanshu Li, Qingyu Yin, Chak Tou Leong, Zhilin Wang, Yanxu Chen, Rongduo Han, Sunbowen Lee, and Yi R. Fung. On the geometry of on-policy distillation, 2026. URL <https://arxiv.org/abs/2606.07082v3>. Version 3, June 14, 2026.
- Yuqiao Wen, Zichao Li, Wenyu Du, and Lili Mou. f -divergence minimization for sequence-level knowledge distillation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 10817–10834. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.acl-long.605. URL <https://aclanthology.org/2023.acl-long.605/>.
- Xiaomi LLM-Core Team. MiMo-V2-Flash technical report, 2026. URL <https://arxiv.org/abs/2601.02780v2>.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388v1>.
- Guo Yu, Wenlin Liu, Yulan Hu, Hao-Xuan Ma, Jun-Peng Jiang, and Han-Jia Ye. Dense supervision, sparse updates: On the sparsity and geometry of on-policy distillation, 2026. URL <https://arxiv.org/abs/2606.13657v3>. Version 3, July 30, 2026.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. DAPO: An open-source LLM reinforcement learning system at scale, 2025. URL <https://arxiv.org/abs/2503.14476>.

A EXPERIMENTAL PROTOCOL AND RESULTS

A.1 MODEL, DATA, AND TRAINING PROTOCOL

The main-study student is Qwen3-1.7B (Yang et al., 2025); training uses seeds 42, 43, and 44. The four teachers were trained by reinforcement learning from the same initial model, specializing in mathematics, code, instruction following, and science, respectively. Student and teachers share the tokenizer and native chat template with thinking disabled. The experiment records identify the model revisions and weight files.

AdamW configurations use learning rate 2.5×10^{-7} with $(\beta_1, \beta_2) = (0.9, 0.98)$, $\epsilon = 10^{-8}$, no weight decay or schedule, and global norm-1 clipping. For the main-study SGD runs, the learning-rate choice follows the empirically optimal setting reported by Mukherjee et al. (2026, Appendix B), supported by their analysis of AdamW’s effective per-parameter learning rates and an SGD learning-rate sweep. Training sampling uses temperature 1 and top- $p = 1$, with a 2,048-token prompt cap and 4,096-token response cap. One fresh response is sampled per prompt. Each optimizer update aggregates 64 responses; domain allocation is specified below.

The multi-teacher setup allocates 16 prompts to each domain per update and uses target domain weights 1/4. The single-teacher setup allocates all 64 prompts to the same domain. I64 uses the top-64 objective in Equation 3; I16 uses $k = 16$. Comparisons within each optimizer share initialization, teachers, and prompt schedules. The vocabulary-supervision comparison also holds response averaging and optimizer settings fixed within each training setting.

A.2 EVALUATION AND SCORING

Each main-study capability suite contains MATH-500 (500 questions) (Hendrycks et al., 2021), LiveCodeBench v6 (1,055 questions) (Jain et al., 2024), IFBench strict (300 questions) (Pyatkin et al., 2025), and GPQA Diamond (198 questions) (Rein et al., 2023). Mathematics, code, and instruction following use one greedy response per question. GPQA uses four responses per question at temperature 1 and top- $p = 1$. Its score is the mean correctness over all four responses, then over questions: average@4 accuracy. Evaluation permits up to 32,768 response tokens, bounded by the remaining configured 32,768-token context. Each complete suite contains 2,053 questions and 2,647 responses per model evaluation.

The expanded development grid comprises updates of 50/100/250/500 for seven Adam trajectories and 100/250/500 for three SGD trajectories, plus the initial student evaluation. Appendix G.1 describes additional test sets for the same four domains. Table 1 gives the complete joint-training endpoint scores. Figure 4(c) compares single-teacher and joint-training trajectories.

The four RL teachers were evaluated separately on their respective domain benchmarks using the same prompt and decoding protocol. These scores quantify each teacher’s performance on its target domain.

A.3 CAPABILITY UNCERTAINTY

The model-based sampling intervals in Table 1 estimate finite-question uncertainty using historical paired-question variance calibration. The calculation uses 100,000 Gaussian parametric-bootstrap replicates (seed 1042), independent domains, and question counts of 500/1,055/300/198 for math-ematics/code/instruction following/GPQA. Domain-specific variance ratios κ are calibrated from historical paired-question intervals relative to independent Bernoulli variances, with score-lattice variance floors. GPQA retains 198 question clusters. Sensitivity checks using $\min(2\kappa, 1)$ and the independent reference $\kappa = 1$ widen the intervals; all evaluated pairwise endpoint contrasts include zero under each of these variance assumptions.

B LENGTH-WEIGHTING DERIVATION

Fix a rollout batch, its valid-token masks, and domain loss weights λ_d . The PG log-ratio coefficients are treated as constants during backpropagation. For $n_d = |\mathcal{B}_d|$, let $\bar{T}_d = n_d^{-1} \sum_{i \in \mathcal{B}_d} T_i$ and

$\bar{g}_d = n_d^{-1} \sum_{i \in \mathcal{B}_d} g_i$. Define the empirical covariance using the same $1/n_d$ normalization:

$$\text{Cov}_d(T, g) = \frac{1}{n_d} \sum_{i \in \mathcal{B}_d} (T_i - \bar{T}_d)(g_i - \bar{g}_d). \quad (10)$$

Expanding gives $n_d^{-1} \sum_i T_i g_i - \bar{T}_d \bar{g}_d$. Dividing by $\bar{T}_d$ gives Equation 6. GT combines the resulting domain gradients using observed token shares; DT combines them using λ_d ; DR combines $\bar{g}_d$ using λ_d . Multiplying each domain’s globally averaged terms by its target weight divided by its observed token share therefore gives DT exactly.

B.1 FIXED-BATCH GRADIENT COMPARISON

At M-PG checkpoints 100 and 250, three cached response batches (seeds 1042, 1043, and 1044) each contain eight responses per domain. Responses use the training prompt format and 4,096-token response cap. For each batch, we compute each response’s mean I64 gradient once at the saved BF16 model values using FP32 computation, then aggregate those gradients under DR, DT, and GT. All three averaging rules use the same responses, token masks, teacher scores, and model coordinates. Each comparison starts from the same student parameters and Adam first and second moments.

Across the six batches, the DR–DT raw-gradient cosine ranges from 0.40 to 0.91 (mean 0.68; Figure 1b). The domain-wise covariance decomposition is checked in FP64 for all 24 batch-domain groups; the largest relative residual is 1.53×10^{-15} . This numerical check verifies the aggregation identity. The measured cosine differences show that response length and gradient direction are correlated in these batches.

The optimizer interventions in Figure 2 reuse these six batches. Each aggregate gradient is globally clipped at norm 1 and applied to the same saved FP32 Adam state for that batch. The saved-state branch retains both moments; the reset- m branch zeros the first moment while retaining the second moment and optimizer step counter; reset- (m, v) zeros both moments while retaining the step counter. The momentum-free SGD control follows the negative gradient and is rescaled separately for DR, DT, and GT to match the FP32 Adam step norm for that averaging rule. After each FP32 update, the model parameters are cast to BF16 and evaluated on one held-out response per domain. Evaluation uses full-vocabulary reverse KL; the experiment archive also retains the I64 surrogate used for training. Each comparison applies a single optimizer step to a checkpoint copy.

C PARAMETER AND OPTIMIZER MEASUREMENTS

C.1 COORDINATES AND RECORDED QUANTITIES

Training measurements separately record gradients before and after clipping, FP32 optimizer steps, and cumulative parameter changes in the FP32 master weights and BF16 model weights. Cumulative changes are measured from the same initialization. All global statistics use unique model coordinates, excluding vocabulary padding and duplicate entries for tied weights. Per-layer statistics and absolute-threshold sensitivities are retained in the experiment archive. For an absolute threshold τ , sparsity is the fraction of coordinates with magnitude at most τ ; the figures show the fraction exceeding τ . Energy concentration uses squared Euclidean norm and ranks coordinates by absolute change.

For checkpoint-based measurements, the exported checkpoint contains FP32 master values, BF16 model values, first and second moments, optimizer hyperparameters, and step counters. Student and teacher values are verified against their saved models in the same precision. Forward and backward passes evaluate these values in FP32 with TF32 disabled. All proposed steps are computed from the same saved state with a non-fused Adam implementation. The online training optimizer uses its own fused implementation.

We compare three Adam states at the same checkpoint. *Saved* retains both moments and the step counter. *Reset m* zeros the first moment while retaining the second moment and step counter. *Fresh* zeros both moments and resets the step counter. Global norm clipping at 1 is applied before the Adam calculation, and weight decay is zero. The plotted controls use two cached response batches;

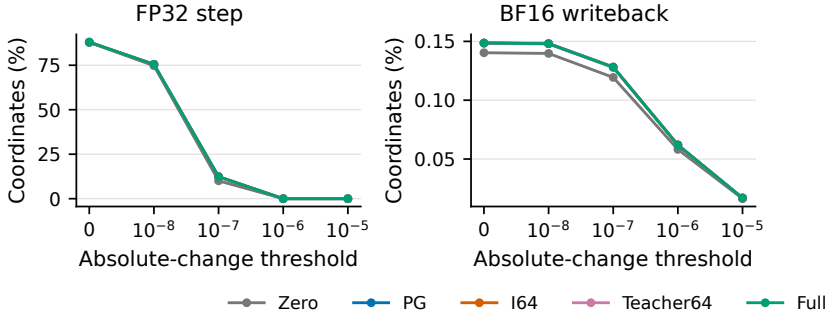


Figure 7: **Update sparsity depends on the numerical threshold.** FP32 steps and BF16 parameter changes; lines show batch means and shading their range.

the zero-gradient control is evaluated once. For the zero-gradient branch, the saved first moment continues to decay and move the master weights. Casting the saved FP32 master weights to BF16 without an optimizer step exactly reproduces the saved model. Figure 7 shows how the threshold changes the measured fraction of updated coordinates.

Setting the current gradient to zero while retaining the trained Adam moments changes 0.14% of BF16 coordinates, compared with 0.15% for PG. Resetting the first moment gives 0.05% for PG; resetting both moments and the step counter gives 0.51%. The parameters changed by Adam depend jointly on the current gradient, accumulated moments, and BF16 rounding. These controls supplement the update-sparsity measurements in Sections 4 and 5.

Figure 6(b,c) displays these BF16 controls. The full FP32 and BF16 measurements are retained in the experiment archive.

C.2 CACHED RESPONSES, GRADIENT OVERLAP, AND GRADIENT NORMS

These raw-gradient diagnostics use the M-PG/100 student, whereas Figure 3(b,c) measures teacher-conditioned optimizer steps at M-PG/500. Batches use seeds 42 and 43, one response per domain per batch, a 2,048-token prompt cap, and a 256-token generation cap. All eight responses reach the cap. Four response positions are sampled without replacement per response. Each response’s four prefixes receive equal weight. For domain-specific inputs, each teacher scores the response from its own domain; for shared inputs, each teacher scores all four responses with equal domain weights. A third control evaluates every teacher on every input domain separately, keeping four prefixes per gradient. These conditions reuse the same responses, allowing paired comparisons within each batch.

Let $S_d(\rho)$ contain the largest-magnitude ρ fraction of teacher d ’s gradient or update coordinates, with stable coordinate-index tie breaking. Jaccard overlap $J(d, e) = |S_d(\rho) \cap S_e(\rho)| / |S_d(\rho) \cup S_e(\rho)|$ measures shared locations; cosine also captures signs and magnitudes (Yu et al., 2026). On domain-specific inputs, the first batch’s instruction-following PG gradient has norm 6.13×10^{-9} . Cosine measures directional opposition, while the gradient norm determines its magnitude. Independent uniform sets of fraction ρ have expected-intersection to expected-union ratio $\rho / (2 - \rho)$, approximately 0.005 at $\rho = 1\%$.

The largest-magnitude 1% of raw-gradient coordinates give mean Jaccard overlaps of 0.095 for PG and 0.100 for I64 on domain-specific inputs, averaging the two batches and six teacher pairs. Their ranges are 0.074–0.108 and 0.074–0.118. When every teacher scores all four domain responses, the respective means are 0.38 and 0.46. The teacher-by-input-domain comparison below uses four prefixes per gradient.

Holding the teacher fixed while changing the input domain gives mean gradient cosines of 0.006 for PG and 0.007 for I64. Holding the input domain fixed while changing teachers produces both aligned and opposed gradients. For domain-specific inputs the mean cosines are -0.002 and 0.008 , respectively, alongside partial coordinate overlap. Figure 8 shows how input choice affects the teacher-overlap measurements in Section 4.

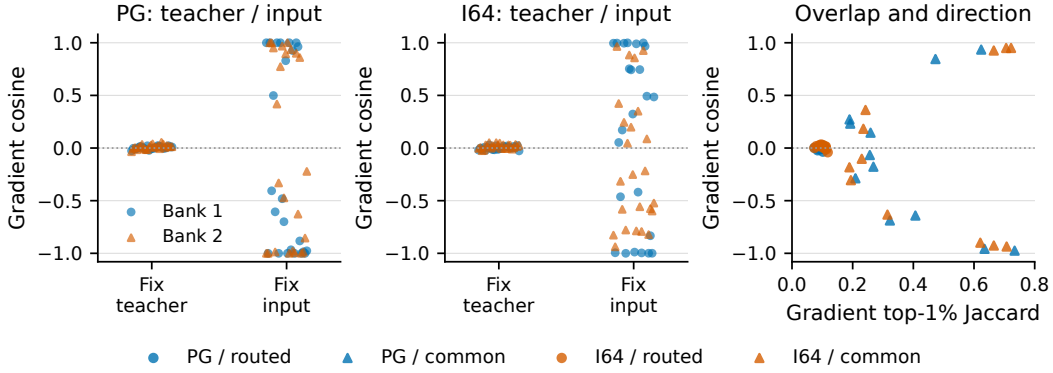


Figure 8: **Gradient overlap depends strongly on the input domain.** Left/center: teacher-by-input-domain controls. Right: top-1% gradient Jaccard versus cosine on domain-specific or shared inputs.

D VOCABULARY SUPERVISION AND COVERAGE

Single-step comparison using four response batches. The M-PG/100 row of Figure 4(a) uses four cached response batches with seeds 2042–2045. Each batch contains one response per domain. Generation uses the training prompt format, a 2,048-token prompt cap, and a 4,096-token response cap. The 16 response lengths range from 81 to 4,096 tokens; two responses reach the cap. Six positions per response give 96 prefixes. PG actions are independently sampled from the student at these fixed prefixes. All losses use the same prefixes, equal domain weights, and equal response weights within each domain. Gradients are globally clipped at norm 1 before applying the same saved Adam state.

The loss and Adam implementations match the two-batch controls in Appendix C. Appendix G.3 reports the actual I64 intersection coverage at all four student checkpoints.

Implementation details. For I64 (Equation 3), candidate sets and teacher scores are fixed at roll-out. Each student forward recomputes current probabilities at the saved candidate positions and refreshes the detached coefficients. PG also refreshes its detached log-ratio coefficients and uses no advantage or policy-ratio clipping.

Teacher-top-64 reference objective. The teacher-top-64 diagnostic uses Equation 8; its correction and distinction from I64 are described in the main text. The I16 measurements cover task performance and online coverage.

Differentiating Equation 2 gives

$$\nabla_{\theta} \ell_{\text{full}} = \sum_{v \in \mathcal{V}} p_{\theta}(v) \left(\log \frac{p_{\theta}(v)}{q_d(v)} + 1 \right) \nabla_{\theta} \log p_{\theta}(v) = \mathbb{E}_{a \sim p_{\theta}} \left[\log \frac{p_{\theta}(a)}{q_d(a)} \nabla_{\theta} \log p_{\theta}(a) \right] \quad (11)$$

using $\sum_v \nabla_{\theta} p_{\theta}(v) = 0$.

E SUPPORTING MEASUREMENTS FOR THE MAIN FIGURES

Teacher-update controls. Figure 3(a) compares cumulative parameter changes at update 500 under DR, with additional I64-DT/GT rows; paired points distinguish FP32 and BF16 for each statistic. Panels (b,c) start from the same M-PG/500 student parameters and Adam moments. With domain-specific inputs, each teacher scores responses from its own domain; with shared inputs, all teachers score the same prefixes. Each of four cached response batches contains six teacher pairs. Small points summarize pairs within a batch, and large points average batches. Equation 7 subtracts the Adam step with the current gradient set to zero before computing FP32 update cosines. SGD and reset- m steps are individually rescaled to the norm of the Adam step with trained moments; reset- m retains the second moment and step counter. These calculations use non-fused PyTorch updates on copies of saved checkpoints.

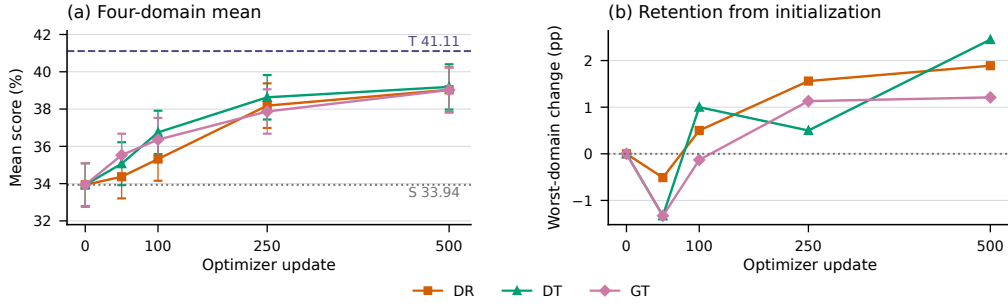


Figure 9: **Mean performance gains can coexist with a regression in an individual domain.** (a) Four-domain mean $\pm$ std; S/T mark the initial student/teacher. (b) Worst-domain change from initialization; zero means no regression. Lines connect reported point estimates.

Vocabulary-supervision controls. Figure 4(a,b) uses four cached response batches at each of four student checkpoints: M-PG and M-I64-DR at updates 100 and 500. Losses use identical prefixes and Adam moments within each checkpoint, while responses can differ between checkpoints. Each checkpoint uses 96 prefixes across the four batches. Panel (a) reports batch means. In panel (b), points are individual batches, lines connect PG batch means, and shading spans the I64 measurements. The direction measurements cover PG, I64, and full-vocabulary supervision; I16 appears in the end-to-end training results. Panel (c) compares S-I64, M-I64, and M-I16 with PG at the same task, teacher setting, and update. Appendix D gives the sampling protocol for the M-PG/100 comparison.

E.1 ADDITIONAL OPTIMIZER AND SAMPLING CONTROLS

Figure 2 uses the six cached response batches and training clipping records described in Appendix B.1. In panel (a), boxes mark the interquartile range and whiskers the 5th–95th percentiles; labels give the fraction of training updates clipped. *Reset- m* and *reset- (m, v)* retain the optimizer step counter. Panel (c) averages four domain probe losses within each batch; error bars show sample standard deviations over the three batches at each checkpoint.

Figure 6(a) uses three M-PG/100 response batches, with a 4,096-token response cap, containing 48 responses and 288 prefixes. Twelve fixed subsets of prefixes each receive two independent draws of 1, 16, or 64 PG actions per prefix. I64 is evaluated once per subset. Increasing the sample count raises the mean PG gradient cosine to full-vocabulary KL from 0.54 to 0.87 and 0.97; I64 exceeds 0.999. Panels (b,c) use the separate two-batch experiment with a 256-token response cap (Appendix C); its zero-gradient control is evaluated once. *Reset- m* retains the second moment and step counter, whereas fresh Adam resets both moments and the counter. The sampling and moment-reset comparisons each use their own response batches.

Cumulative parameter changes. Figure 10 shows the trajectories underlying the update-500 summary in Figure 3(a).

F PROMPT EXPOSURE AND CANDIDATE-SET WEIGHTS

When each domain loss is averaged separately, its prompt quota and loss weight control different quantities. Increasing the quota at fixed λ_d changes the number of sampled responses and the variance of the gradient estimate, without directly increasing the domain’s expected contribution to the total gradient. Mathematics-only runs receive four times as many mathematics prompts per update as joint runs.

Candidate-set coverage also scales the distillation gradient. For the student candidate set S and its intersection I with the teacher set in Equation 3, the retained student weights sum to $r(h) = p_\theta(I | h)/p_\theta(S | h)$. At a fixed prefix with a nonempty intersection and detached coefficients, the I64 gradient equals $r(h)$ times the gradient obtained by renormalizing the same weights over

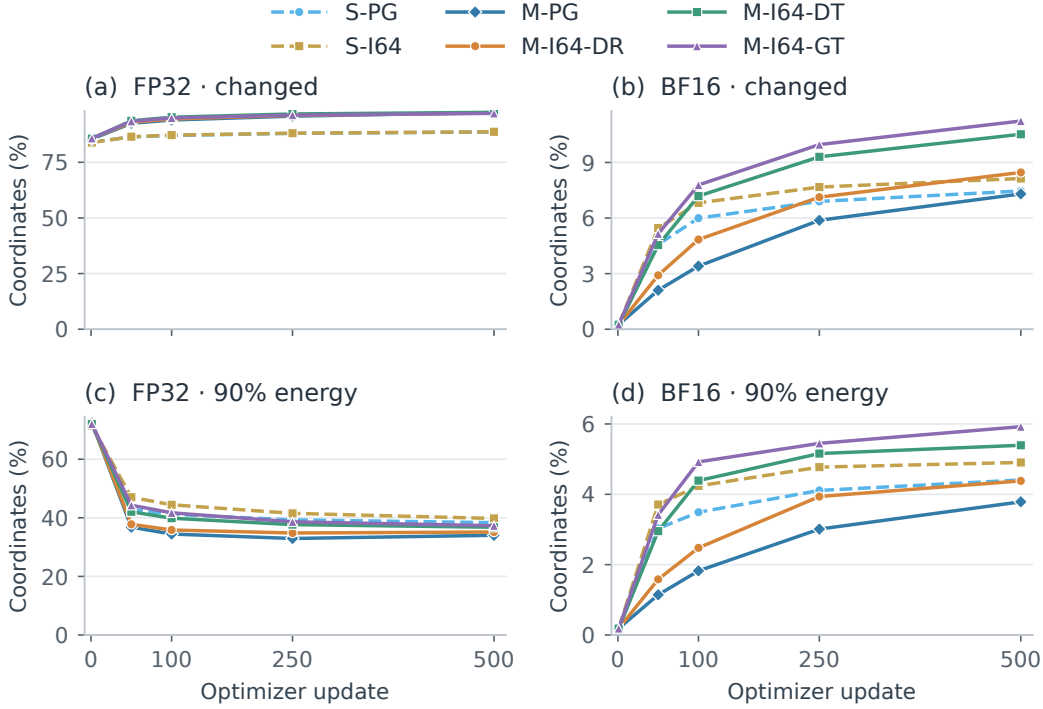


Figure 10: **Cumulative BF16 changes remain much sparser than FP32 changes.** PG/I64 trajectories under DR in single-teacher (S) and joint (M) training, with joint I64-DT/GT added: (a,b) nonzero coordinate fractions; (c,d) coordinate fractions containing 90% of squared displacement from initialization. Lines connect measured updates 1, 50, 100, 250, and 500; dashed lines denote S.

I. Coverage therefore changes gradient scale across prefixes even under equal response weighting. Measuring the retained mass and empty-intersection rate by domain can reveal unequal coverage under a fixed candidate size.

G ADDITIONAL TEST RESULTS AND ROBUSTNESS CHECKS

G.1 ADDITIONAL TEST SETS FOR THE SAME FOUR DOMAINS

The ten joint-training configurations use the development benchmarks and checkpoint grid in Appendix A. The following analyses evaluate selected checkpoints on separate test questions, measure candidate-set coverage, and check sensitivity to known training–evaluation overlap.

The separate-test means in Table 1 use 256 additional questions per domain: 1,024 questions and 1,792 responses per model. Mathematics and code come from the Skywork-OR1-RL-Data mathematics and TACO-based code pools; instruction following uses Nemotron-RL-instruction_following; science uses the Nemotron-RL-knowledge-mcqa validation split restricted by a fixed natural-science keyword filter. Questions are the first eligible unique candidates in the fixed source order. Normalized exact-match and lexical near-duplicate checks remove questions matching the original training bundle or known evaluation pools.

Scoring uses mathematics answer verification, code unit tests, instruction-following constraints, and science multiple-choice accuracy. Mathematics, code, and instruction following use greedy decoding; science uses four responses per question at temperature 1. The context budget is 32,768 tokens, with thinking disabled. The four-domain mean weights domains equally. Question-paired bootstrap intervals quantify finite-test uncertainty.

G.2 CHECKPOINT SELECTION BEFORE TEST EVALUATION

We apply the three selection rules in Section 6 to the development grid in Appendix A. The grid contains updates 50/100/250/500 for seven Adam trajectories and 100/250/500 for three SGD trajectories. The retention tolerance is two percentage points per domain relative to the initial student. Both development rules keep the initial student unless the selected equal-domain mean improves by at least one percentage point. Both development-based rules choose update 250 for PG-DT/Adam and update 500 for the other nine trajectories. The separate PG-DT/Adam update-250 test evaluation allows direct comparison with its update-500 endpoint.

The development grid contains 37 trained-checkpoint evaluations along the same ten trajectories, including the ten endpoints reported in Table 1.

The retention-constrained and maximum-mean rules select the same checkpoint in every trajectory. On the additional test sets, the update-250 PG-DT/Adam checkpoint scores 0.37 percentage points below its update-500 endpoint in four-domain mean. The endpoint therefore has the higher separate-test mean in this comparison. PG-DR/Adam has two intermediate development checkpoints outside the retention tolerance: GPQA falls from 32.20% initially to 28.03% at update 50 and 30.18% at update 250, before reaching 35.97% at update 500. The learning curve captures these temporary regressions and their subsequent recovery.

G.3 PROBABILITY MASS RETAINED BY THE CANDIDATE INTERSECTION

For each of four student checkpoints (M-PG and M-I64-DR at updates 100 and 500), four cached response batches provide one response per domain and six sampled positions per response: 96 prefixes per checkpoint. Each response is scored by its domain teacher. We measure $p_\theta(I | h)$ for the actual intersection $I = S \cap T$ and the retained fraction $r = p_\theta(I | h)/p_\theta(S | h)$.

Mean student probability mass in the actual student–teacher top-64 intersection exceeds 99.9% at each of the four checkpoints. The minimum over the 384 sampled prefixes is 96.55%, and no intersection is empty.

G.4 SENSITIVITY TO KNOWN MATHEMATICS OVERLAP

The fixed Qwen training pool contains five records matching four known MATH-500 problems, including variants in wording or answer format. The affected MATH test identifiers are counting-and-probability problem 430 and intermediate-algebra problems 1849, 2146, and 582. We remove these four questions from the existing per-question evaluation results, retaining the original 500-question results as the main protocol.

Removing the four questions changes the four-domain means by approximately 0.09–0.16 points and can reorder near-tied configurations. PG-DT/Adam and PG-GT/Adam both round to 38.60% before removal, but reach 38.70% and 38.75%, respectively, afterward. PG-DR/SGD retains the highest mean among the reported checkpoints. MATH-496 denotes this retained subset of MATH-500.

G.5 TEACHER-UPDATE OVERLAP AND DISTRIBUTIONAL SIMILARITY

Figure 11 supplements the zero-gradient baseline subtraction at M-PG/500 using the controls in Appendix E. It compares PG and I64 under Adam with trained moments, reset- m , and norm-matched SGD. Shared inputs hold student prefixes fixed between teachers; domain-specific inputs change both the teacher and the input domain. Each of four batches contains six teacher pairs.

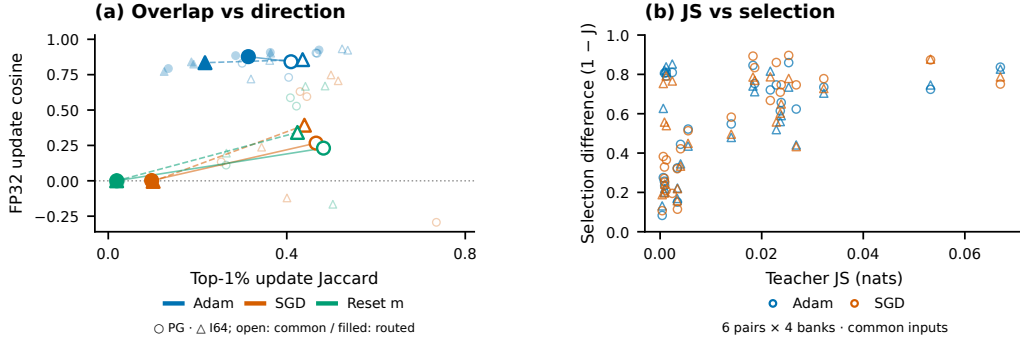


Figure 11: **Coordinate overlap and update direction measure different properties.** (a) Top-1% FP32 update-set Jaccard versus cosine; small/large points show batch/overall means. (b) Teacher-distribution JS divergence versus $1 - \text{Jaccard}$ on shared inputs; each point is a teacher pair within a batch.

G.6 ONLINE COVERAGE AND PER-DOMAIN NORMALIZATION EFFECTS

Online I16 coverage. For I16-DR, each of 500 updates has 16 logged slices of four responses. Each update averages the masked, response-averaged summaries with equal weight per slice. Candidate sets are fixed at rollout; student probabilities are measured during training. Across 8,000 slices, mean $p_\theta(I)$, r , and $|I|$ are 99.62%, 99.88%, and 15.05. The mean empty fraction is 0.0027%, nonzero in 80 slices across 62 updates. The lowest slice-mean mass is 86.67% and largest slice-mean empty fraction 1.85%; these extremes refer to slice means.

Per-domain probe losses. We disaggregate Figure 2(c), using the same six batches and twelve branches per batch (Appendix B.1). Each batch has one held-out response per domain. At update 250, saved-state Adam under DR/DT/GT increases IF KL by $0.362/0.293/0.272 \times 10^{-3}$, while the four-domain changes are $0.041/-0.002/-0.027 \times 10^{-3}$. Thus the mean reduction under DT/GT coexists with an instruction-following increase.

H SMOLLM3 DIAGNOSTICS AND CAPABILITY EVALUATION

H.1 CHECKPOINTS, RESPONSES, AND WEIGHTING

We use the SmolLM3-3B MixSFT student, three domain-specific RL teachers, and exported 500-update students for I64-DR/DT/GT and PG-DR/SGD. The model and data revisions, export hashes, and diagnostic source snapshots are recorded with the measurements. The reported I64 training configurations use Adam with learning rate 2.5×10^{-7} ; PG uses SGD with learning rate 0.0025. The capability comparison uses a new shared evaluation protocol.

Diagnostic prompts come from the frozen Open-MOPD data revision 9e897efe, with unique raw messages shuffled within mathematics, code, and instruction following. Coverage, normalization, and probe prompt splits are disjoint within this diagnostic sample. We preserve dataset messages and disable thinking in the chat template. Generation uses temperature 1, top- $p = 1$, a 2,048-token prompt cap, and a 4,096-token response cap. Each natural coverage bank contains 384 responses, 128 per domain, with up to eight uniformly spaced positions per response: 3,053 prefixes for MixSFT and 3,058 for DR500. Additional students score the same 3,058 DR500 prefixes without regenerating responses.

Coverage statistics assign equal weight to domains, responses within domains, and audited positions within a response. Mean-mass intervals resample responses independently within each domain 2,000 times using seed 1042/1043/1044. MixSFT and DR500 use their own natural banks; the remaining students use the DR500 bank. The minimum I64 mass on DR500’s natural bank is 1.7×10^{-8} ; DT500 and GT500 each have one empty intersection in the common-bank audit.

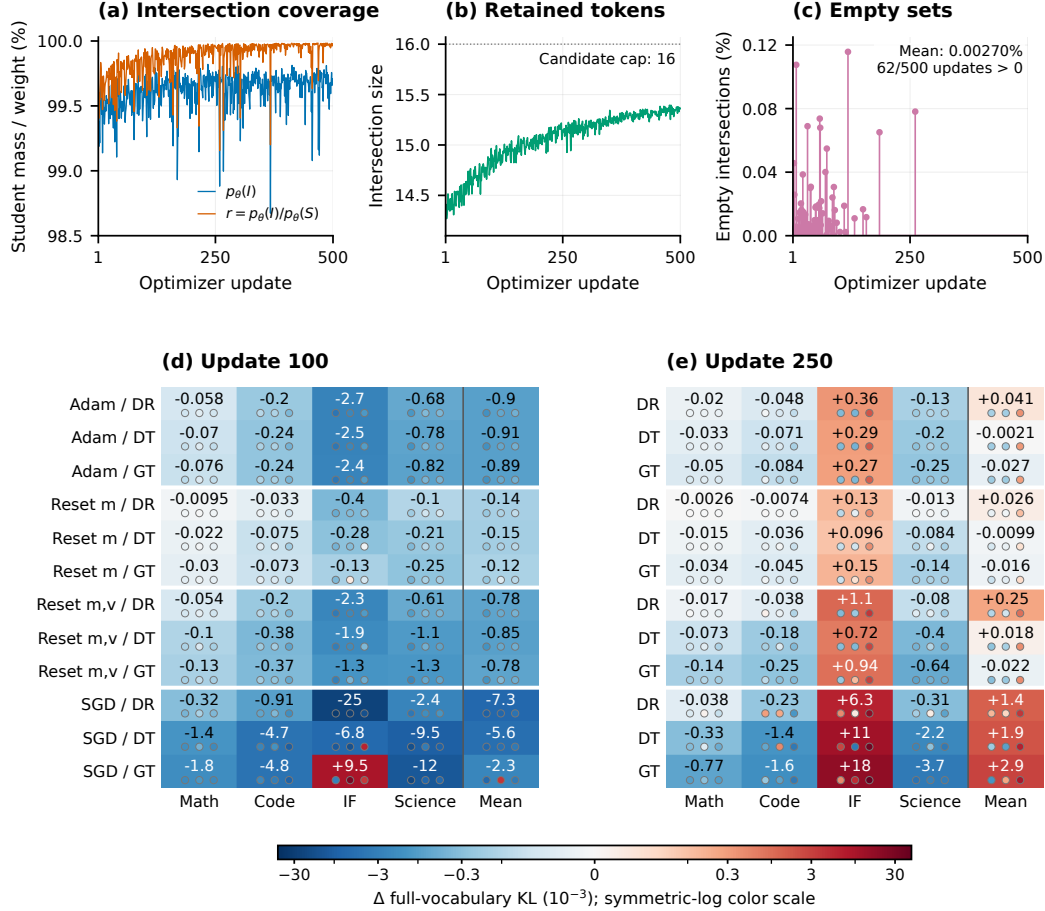


Figure 12: **High average coverage and low mean probe loss can conceal exceptions.** (a–c) I16 per-update means of 16 logged slices, without temporal smoothing; the mass axis spans 98.5–100%. (d,e) Full-vocabulary KL after minus before BF16 writeback, in units of 10^{-3} . Cells show three-batch means; dots show batches 1042–1044 from left to right. The shared symmetric-log color scale is linear within ± 0.1 . SGD is matched to each rule’s Adam step norm.

H.2 FROZEN SUBSETS AND GRADIENT COMPARISONS

Within each domain we select representative prefixes, prefixes above its 75th entropy percentile, and prefixes with I64 mass below 0.99. Selection allows overlap between strata and uses up to 32 distinct prompts per domain. The low-coverage code stratum has 21 eligible prompts, yielding 85 distinct prefixes overall. Each stratum is divided into three batches. Small candidate budgets $k = 4, 8$ are selected by a coverage-only rule.

The BF16 coverage audit sorts a top-128 list and takes its first k entries, whereas gradient diagnostics recompute top- k directly in FP32 at the exported BF16 parameter values. Student forward/backward passes use FP32; cached teacher targets use BF16 forward passes followed by FP32 log-softmax. One of the 85 frozen low-coverage prefixes crosses 0.99 during the FP32 diagnostic and remains in its original stratum. The FP32 diagnostic mean I64 masses are 99.05%, 95.68%, and 93.12% for the representative, high-entropy, and low-coverage groups.

Full KL, PG, and intersection objectives use the definitions in Appendix D. Empty intersections produce zero loss. We retain zero- and small-norm cases and report vector differences in addition to cosine. The plotted relative error is $\|g_{I_k} - g_{\text{full}}\|/\|g_{\text{full}}\|$, which measures the vector difference relative to the full-vocabulary gradient norm.

H.3 OPTIMIZER STATE, SAMPLING, AND NUMERICAL PRECISION

Normalization uses three separate batches, each with eight responses per domain, and the true response lengths for GT/DT weights. Raw gradients, fresh Adam, and SGD start from independent copies of the same exported parameters. Fresh Adam starts with both moments zero and performs its first step; SGD uses learning rate 0.0025, zero momentum, and norm-1 clipping. Fresh Adam is distinct from the Qwen reset- m control, which retains the trained second moment and step counter.

For all three rule pairs in all three batches, fresh Adam lowers cosine relative to the raw gradients. For example, DT-DR raw cosines are 0.90/0.87/0.59, versus 0.70/0.65/0.41 after fresh Adam.

Conditional PG variance uses three fixed prefixes per group, one per domain, with eight independent action-sampling repetitions. Sample budgets $m = 1, 16, 64$ use nested actions within a repetition. We normalize the total coordinate sample variance by $\|g_{\text{full}}\|^2$ and separately report the error of the eight-repetition mean. The representative-group variance is 1.43/0.57/0.12; the low-coverage-group variance is 4.97/2.57/0.43.

The precision controls begin exactly at parameter values represented by BF16, cast to FP32 for computation.

H.4 CUMULATIVE BF16 CHANGES BY PARAMETER GROUP

The exported MixSFT-to-DR500 comparison contains 3,075,098,624 unique parameter coordinates after accounting for tied embeddings. The 36 transformer blocks each contain 78,123,008 coordinates, the token embedding 262,668,288, and the final normalization 2,048. We compare the two BF16 exports directly. Overall, 10.47% of coordinates change; the largest 1% carry 36.85% of squared displacement, and 5.85% of coordinates carry 90%. Figure 14 retains all parameter groups. Per-group changed fractions use each group’s own coordinate count; displacement shares use total model squared displacement.

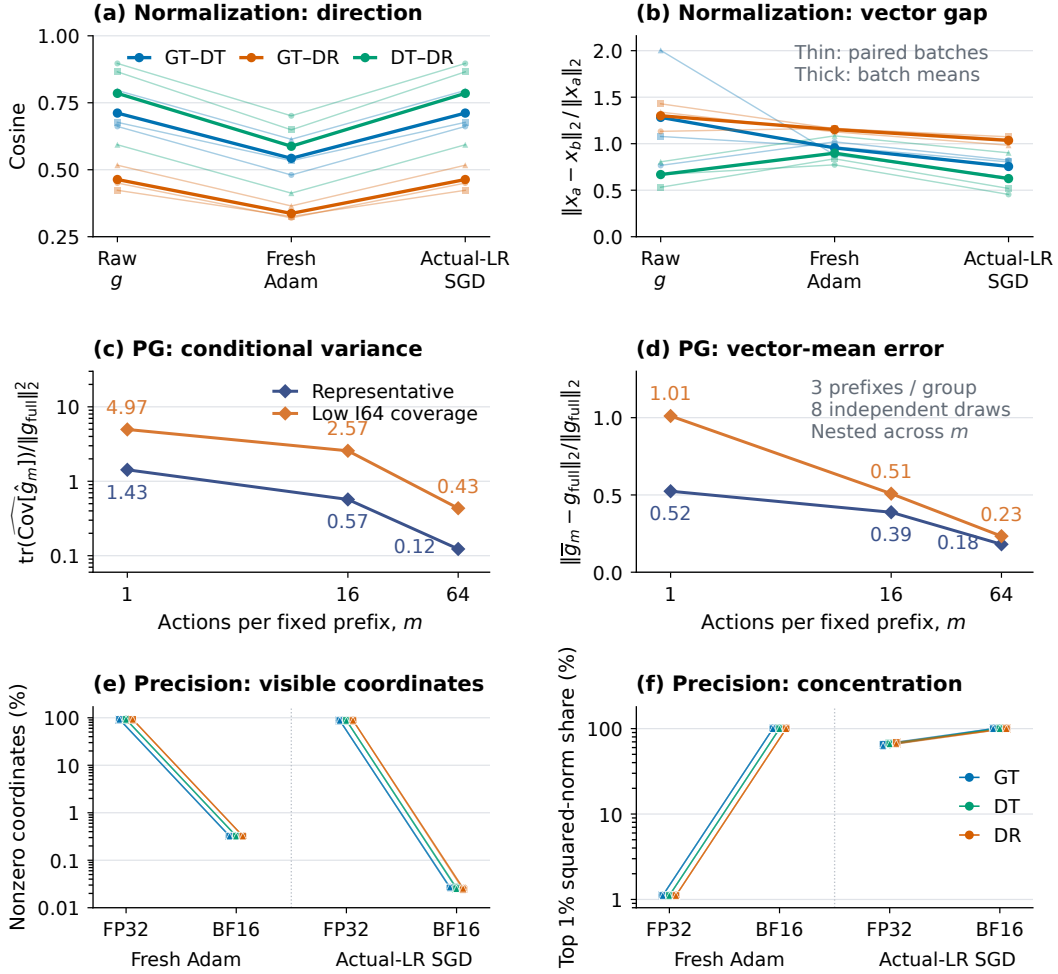


Figure 13: **Local controls separate optimizer, sampling, and precision effects.** (a,b) Three paired SmolLM3 batches (thin) and their means (bold). In (b), x is the gradient or step, normalized by the first named rule. (c,d) Three fixed prefixes per group, eight action repeats; (d) error of the vector mean. (e,f) Each rule/batch pairs FP32 changes with BF16 writeback. Fresh Adam starts at the exported BF16 parameter values with both moments zero.

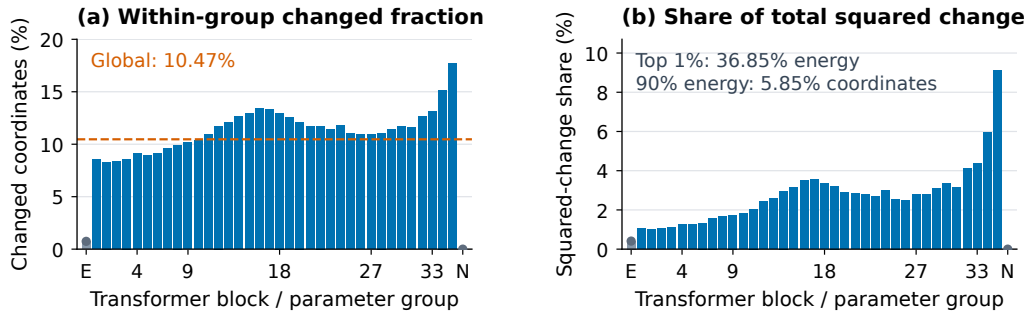


Figure 14: **Cumulative BF16 changes differ across all 38 SmolLM3 parameter groups.** MixSFT to DR500: blocks 0–35, embedding (E), final norm (N). (a) Within-group changed fraction; dashed: global parameter-weighted fraction. (b) Share of total squared displacement. Tied weights count once.

H.5 ENDPOINT CAPABILITY EVALUATION

We evaluate the exported students with shared prompts, scorers, and decoding settings within each benchmark. We use BF16, disable thinking, and allocate a 32,768-token context with the response budget equal to context length minus prompt length. MATH-500, LiveCodeBench v6, and IFBench use greedy decoding; GPQA uses temperature 1 and top- $p = 1$; AIME uses temperature 0.6 and top- $p = 0.95$. Budget-capped outputs remain in scoring. Score and paired-difference intervals use 100,000 question-bootstrap replicates (seed 1042/1043/1044), retaining all responses to each sampled question.

MATH-500 greedy accuracy is 50.60% for MixSFT and 53.80% for update-500 PG-DR/SGD, a change of +3.20 points (95% paired interval $[-0.20, 6.60]$), gaining 45 questions and losing 29. Update-500 I64-DT/Adam changes AIME 2024 pass@8 by +16.67 points ($[0.00, 33.33]$) relative to MixSFT.

Equal endpoint scores can hide different solved questions. AIME pass@8 is the fraction of questions with at least one correct answer among eight samples. MixSFT, update-500 I64-DT/Adam, and update-500 PG-DR/SGD solve 3/30, 8/30, and 5/30 questions on AIME 2024, respectively. On AIME 2025, each solves 4/30. Their average response accuracies are 1.67/3.75/2.08%, and only one solved question is common to all three. MixSFT and PG-DR/SGD also share only one of their four solved questions (Figure 15).

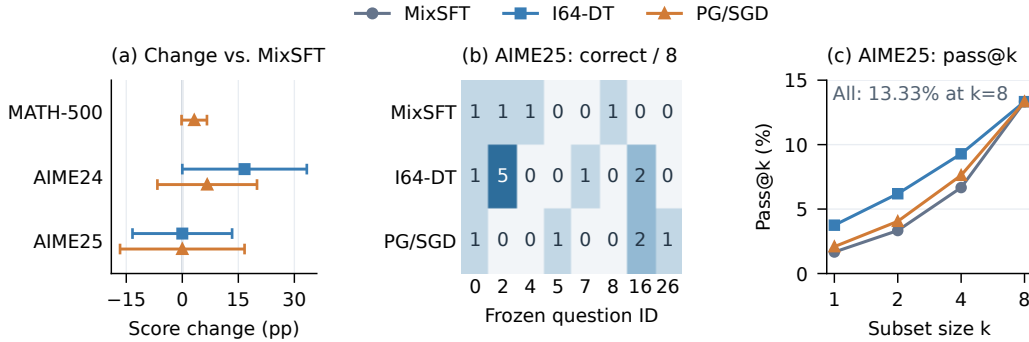


Figure 15: **Equal aggregate scores can conceal different solved questions and sampling outcomes.** (a) Differences from MixSFT with paired question-bootstrap 95% intervals: accuracy for MATH-500, pass@8 for AIME. (b) Correct responses out of eight for every AIME 2025 question solved by any model; the other 22 questions have zero successes. (c) Subset estimates from the same eight responses: $\widehat{\text{pass@}k} = \frac{1}{30} \sum_q [1 - \binom{8-c_q}{k} / \binom{8}{k}]$, where c_q counts correct responses.